





With or Without You: Programming with Effect Exclusion

MATTHEW LUTZE, Aarhus University, Denmark
MAGNUS MADSEN, Aarhus University, Denmark
PHILIPP SCHUSTER, University of Tübingen, Germany
JONATHAN IMMANUEL BRACHTHÄUSER, University of Tübingen, Germany

Type and effect systems have been successfully used to statically reason about effects in many different domains, including region-based memory management, exceptions, and algebraic effects and handlers. Such systems' soundness is often stated in terms of the absence of effects. Yet, existing systems only admit indirect reasoning about the absence of effects. This is further complicated by effect polymorphism which allows function signatures to abstract over arbitrary, unknown sets of effects.

We present a new type and effect system with effect polymorphism as well as union, intersection, and complement effects. The effect system allows us to express *effect exclusion* as a new class of effect polymorphic functions: those that permit any effects except those in a specific set. This way, we equip programmers with the means to directly reason about the absence of effects. Our type and effect system builds on the Hindley-Milner type system, supports effect polymorphism, and preserves principal types modulo Boolean equivalence. In addition, a suitable extension of Algorithm W with Boolean unification on the algebra of sets enables complete type and effect inference. We formalize these notions in the λ_{C} calculus. We prove the standard progress and preservation theorems as well as a non-standard effect safety theorem: no excluded effect is ever performed.

We implement the type and effect system as an extension of the Flix programming language. We conduct a case study of open source projects identifying 59 program fragments that require effect exclusion for correctness. To demonstrate the usefulness of the proposed type and effect system, we recast these program fragments into our extension of Flix.

CCS Concepts: • **Theory of computation → Type theory**; • **Software and its engineering → General programming languages**.

Additional Key Words and Phrases: polymorphic types and effects, effect exclusion, without construct

ACM Reference Format:
Matthew Lutze, Magnus Madsen, Philipp Schuster, and Jonathan Immanuel Brachthäuser. 2023. With or Without You: Programming with Effect Exclusion. *Proc. ACM Program. Lang.* 7, ICFP, Article 204 (August 2023), 28 pages. https://doi.org/10.1145/3607846

1 INTRODUCTION

Conceived more than 30 years ago by Lucassen and Gifford, type and effect systems have successfully been used by programmers and compilers to statically reason about side effects [Leijen 2014; Lucassen and Gifford 1988], checked exceptions [Gosling et al. 1996], and region-based memory management [Tofte and Talpin 1997]. Recently, type and effect systems are again gaining ground

Authors' addresses: Matthew Lutze, mlutze@cs.au.dk, Department of Computer Science, Aarhus University, Aarhus, Denmark; Magnus Madsen, magnusm@cs.au.dk, Department of Computer Science, Aarhus University, Aarhus, Denmark; Philipp Schuster, philipp.schuster@uni-tuebingen.de, Department of Computer Science, University of Tübingen, Tübingen, Germany; Jonathan Immanuel Brachthäuser, jonathan.brachthaeuser@uni-tuebingen.de, Department of Computer Science, University of Tübingen, Tübingen, Germany.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2023 Copyright held by the owner/author(s).
2475-1421/2023/8-ART204
https://doi.org/10.1145/3607846

to establish effect safety in languages with support for algebraic effects and handlers [Bauer and Pretnar 2015; Brachthäuser et al. 2020; Leijen 2014; Lindley et al. 2017].

Such systems' soundness is often phrased in terms of the *absence* of effects. An expression is pure if it does not have any side effects, memory is never allocated into regions that are not live, no exception raised at runtime will be unhandled, and so on. Maybe surprisingly, reasoning about the absence of effects in those systems can only be achieved indirectly: if we want to assert that an expression cannot perform a specific effect, we need to check that this effect is not mentioned in the expression's effect set. This mode of reasoning works well when applied globally, as it is done to prove soundness. It can also be applied locally, as long as the set of effects of an expression is closed: we can manually assert that the closed set does not mention the effect we want to forbid.

However, as soon as a language also supports *effect polymorphism*, local reasoning about the absence of effects breaks down. Effect polymorphism allows programmers to express that the effect of a higher-order function depends on the effects of its function argument [Lucassen and Gifford 1988]. For example, the effect of List.map depends on the effect of its argument. However, since effect-polymorphic functions universally quantify over their effects, we cannot make any assertions about the absence of specific effects.

Reasoning about the absence of effects is practically relevant to establish correctness in real-world programs. We have discovered multiple programming patterns where higher-order functions accept function arguments ("callbacks") which may have any effects *except* for a specific set of effects. For example, (i) an iterator should not modify its underlying collection, (ii) an exception handler should not itself throw an exception, (iii) a file operation should not close the underlying input stream, (iv) an event listener should not block the UI thread. We call these correctness criteria *effect exclusions*. Unfortunately, current programming languages cannot directly enforce effect exclusion statically. Instead, such safety properties are commonly relegated to documentation, even though violations may lead to program crashes, data races, and undefined behavior.

We propose to overcome these issues with a new type and effect system that directly supports effect polymorphism and effect exclusion. We present the $\lambda_{\complement}$ calculus, a lambda calculus with effect operations that features one new term-level construct: "e without F". It specifies that the expression e must not have the effect F when evaluated (but may have any other effects). For example, we can express that calling a function f must not have the Throw effect as follows:

$$\mathsf{f}\,()\ \mathsf{without}\ \mathsf{Throw}$$

The semantics of $\lambda_{\complement}$ enforces that an expression "do Throw(v)" cannot take a step in a context that forbids Throw. In other words, a program that attempts to perform a "forbidden" effect operation gets stuck. To prevent terms from getting stuck, we equip $\lambda_{\complement}$ with a type and effect system that extends Hindley-Milner with a language of *effect set formulas* including union, intersection, and complement. This way, we can express function types that are effect polymorphic but exclude a specific set of effects. Specifically, we can express subtraction using intersection and complement, i.e., $S \setminus T \equiv S \cap T^{\complement}$. The inferred type of the function f above is

$$\forall a, ef.\ \mathsf{Unit} \xrightarrow{ef \setminus \{\mathsf{Throw}\}} a$$

which takes values of type Unit to values of type a with any effect *except* the Throw effect. We prove soundness of the type and effect system for the $\lambda_{\complement}$ calculus, i.e., the standard progress and preservation theorems. We also prove an *effect safety* theorem, which guarantees that if a program is about to perform an effect operation, then that operation is not forbidden by the current stack of "without frames". In other words, forbidden effects will never occur at runtime and the program will not be stuck.

The type and effect system supports complete type inference by an extension of Algorithm W with Boolean unification on sets. An important implementation detail is treating the top element of the Boolean algebra, i.e., the universe of effects, as symbolic during type and effect inference. We implement the proposed type and effect system and the "without" construct as an extension of the Flix programming language. Flix[1] is a functional-first, imperative, and logic programming language that supports algebraic data types, pattern matching, higher-order functions, parametric polymorphism, type classes, and higher-kinded types. The extension enables programmers to define effects, attach them to functions, and use the without construct to forbid an expression from having a specific set of effects.

We conduct a case study of open-source projects identifying 59 program fragments that require effect exclusion to ensure correctness. We observe that violation of these safety criteria can lead to program crashes, race conditions, deadlocks, and even undefined behavior. The case study shows that effect exclusion is a real problem with potentially harmful consequences. We illustrate the usefulness of the proposed type and effect system by recasting these 59 program fragments in our extension of Flix where the exclusion of the specific effects is statically enforced.

Contributions. In summary, the paper makes the following contributions:

- (**Calculus**) We present the $\lambda_{\complement}$ calculus: a minimal lambda calculus with effect operations and a "without" construct that enables programmers to declare that the evaluation of an expression may perform any effect *except* for a specific excluded effect.
- (**Type and Effect System**) We present a type and effect system for the $\lambda_{\complement}$ calculus based on Hindley-Milner extended with union, intersection, and complement effects. The system is expressive, fine-grained, and supports complete type and effect inference.
- (**Meta Theory**) We prove progress and preservation for the $\lambda_{\complement}$ calculus and its type and effect system. We also prove *effect safety*: that execution of an effect operation cannot happen when it is forbidden by the current context.
- (**Implementation**) We implement the type and effect system as an extension of the Flix programming language. We extend Algorithm W with Boolean unification on sets.
- (**Evaluation**) We conduct a case study of code on GitHub and identify 59 program fragments where effect exclusion is required for correctness. We show that these program fragments can be recast in our Flix extension where effect exclusion is statically enforced by the type and effect system. We also implement two libraries that make use of effect exclusion, as well as runnable applications exercising the libraries.

Paper Structure. This paper is structured as follows: In Section 2 we motivate the need for effect polymorphism modulo exclusion by listing several examples of real-world programming patterns. We also informally introduce the proposed type and effect system and show how it can capture these programming patterns. In Section 3 we present the $\lambda_{\complement}$ calculus, including its type and effect system, and key theoretical properties, including the effect safety theorem. In Section 4 we discuss the implementation of the type and effect system in the Flix programming language. In Section 5 we present a case study of real-world programming patterns. Building on the examples shown earlier, we illustrate how these program fragments can be safely recast in Flix. In Section 6 we present two Flix libraries demonstrating effect exclusion and applications using the libraries. In Section 7 we present related work and in Section 8 we conclude.

[1] https://flix.dev/ and https://github.com/flix/flix/

2 MOTIVATION

We motivate our work by first listing several real-world examples of programs that rely on *effect exclusion*, and then demonstrate how these can be safely expressed in our extension of the Flix programming language.

2.1 Real-World Examples

The following is a small but representative selection of real-world examples identified in our case study (Section 5). They illustrate how programmers today are forced to rely on comments to document effect exclusion.

googleapis/google-cloud-go (Go) `firestore/transaction.go`

```
// RunTransaction runs f in a transaction. f should use the transaction
// it is given for all Firestore operations. ...
//
// f must not call Commit or Rollback on the provided Transaction.
func (c *Client) RunTransaction(ctx context.Context,
                                f func(context.Context, *Transaction) error,
                                opts ...TransactionOption) (err error) { ... }
```

In the Google Cloud Client Libraries for Go, there is a function RunTransaction demonstrating an effect exclusion requirement. As the documentation states, the function f passed to RunTransaction may have any effect *except* to commit or rollback the transaction. The documentation does not specify what happens if a function has one of these two effects.

libevent/libevent (C) `include/event2/event.h`

```
/**
  * Iterate over all added or active events in an event loop,
  * and invoke a given callback on each one.
  *
  * The callback must not call any function that modifies the event base,
  * that modifies any event in the event base, or that adds or removes
  * any event to the event base.  Doing so is unsupported and will lead
  * to undefined behavior -- likely, to crashes.
  * */
EVENT2_EXPORT_SYMBOL
int event_base_foreach_event(struct event_base *base,
                             event_base_foreach_event_cb fn, void *arg);
```

In libevent, a popular event notification library, there is a C header file include/event2/event.h requiring effect exclusion. The documentation states that the callback *fn* must not modify the "event base"; e.g. by adding or removing events. Doing so could lead to undefined behavior or program crashes.

decred/dcrd (Go) `connmgr/connmanager.go`

```
// ForEachConnReq calls the provided function with each connection request
// known to the connection manager, including pending requests. ...
//
// NOTE: This must not call any other connection manager methods during
```

```
// iteration or it will result in a deadlock.
//
func (cm *ConnManager) ForEachConnReq(f func(c *ConnReq) error) error {
    /* ... */
}
```

In Decred, a blockchain-based cryptocurrency, the file connmgr/connmanager.go contains the function ForEachConnReq, where the documentation states that the callback f may have any effect *except* to call methods on the connection manager. If it does, the documentation states that it could lead to a deadlock.

akka/akka (Scala) `akka-stream/src/main/scala/akka/stream/impl/SubscriberManagement.scala`

```
trait SubscriberManagement[T] extends ... {
    /**
     * called when we are ready to consume more elements
     * from our upstream MUST NOT call pushToDownstream
     */
    def requestFromUpstream(elements: Long): Unit
}
```

In Akka, a popular actor-framework for Java and Scala, the file SubscriberManagement.scala contains a trait where the documentation specifies that implementations of the requestFromUpstream *must not* call the pushToDownstream method.

qos-ch/logback (Java) `logback-core/src/main/java/ch/qos/logback/core/encoder/Encoder.java`

```
/**
 * This method is called prior to the closing of the underlying
 * {@link OutputStream}. Implementations MUST not close the underlying
 * {@link OutputStream} which is the responsibility of the owning appender.
 */
void close() throws IOException;
```

In LogBack, a popular logging framework for Java intended as a replacement for log4j, file Encoder.java contains the interface that specifies that the close method *must not* close the output stream associated with the object.

Based on these examples, and our much larger case study (c.f. Section 5), we observe:

> **Observation:** Programmers often write code whose correctness *depends on the exclusion of a specific effect* while permitting any other effect.

In particular, programmers use such effect exclusions to prevent:

- resources from being inadvertently released,
- deadlocks resulting from locks being acquired cyclically,
- locks from being inadvertently unlocked or re-locked,
- error handlers from throwing exceptions themselves,
- iterators from modifying their underlying collection,
- infinite recursion or looping behavior, and
- re-entrance issues where a function must not enter itself.

Unfortunately, current programming languages do not allow programmers to express such invariants. Consequently, they are not checked by the compiler and programmers must learn of them from the documentation (or from bitter experience). As we have seen, violation of these invariants can lead to program crashes, deadlocks, or even undefined behavior. In this paper, we propose to overcome these issues with a new type and effect system that supports effect polymorphism, effect exclusion, and complete type and effect inference.

2.2 The Proposed Type and Effect System

We now illustrate the proposed type and effect system. We begin with some simple examples working our way to the more complex. We show type and effect signatures to explain the intuition, but want to stress that all types and effects can be inferred. All program fragments shown here are valid in our extension of Flix.

*Example 2.1 (**Purity**).* We can express that a function is pure (i.e. has no side effects):

```
def add(x: Int32, y: Int32): Int32 \ { } = ...
                                  // ^^^ empty effect set
```

Here the signature of add enforces that its implementation *cannot* have any side-effects.

The Flix standard library has many use cases for enforcing purity. For example, the Eq type class enforces that the eq function is pure. Similarly, the Order type class enforces that the compare function is pure. This ensures that equality and ordering is stable. Purity also ensures that these functions cannot leak implementation details about the collections that rely on them.

*Example 2.2 (**Higher-Order Purity**).* We can also express that a higher-order function requires a pure function argument:

```
def count(f: a -> Bool \ { }, l: Set[a]): Int32 \ { } = ...
                     // ^^^ empty effect set     ^^^ empty effect set
```

Here the signature of count enforces that f *cannot* have any side-effects and also that the implementation of count itself *cannot* have any side-effects.

*Example 2.3 (**Concrete Effects**).* We can express that a function has a specific effect:

```
def sayHello(name: String) : Unit \ { Console } =
    do println("Hello ${name}!")  //  ^^^^^^^ concrete effect
```

Here the signature of sayHello specifies that the function has the Console effect. Knowing and controlling the specific effects of an expression is useful both to the programmer and to the compiler.

*Example 2.4 (**Effect Polymorphism**).* We can express that the effect of a function depends on the effects of its function arguments, i.e. we have *effect polymorphism*:

```
def map(f: a -> b \ ef, l: List[a]): List[b] \ ef = ...
                 // ^^ effect variable         ^^ effect variable
```

Here the signature of map expresses that if f has effect ef then map has effect ef. In particular, if f is pure then the call to map is pure. Note that the signature of map also expresses that map has no other effects than those of f, i.e. map does not cause any effects itself.

*Example 2.5 (**Effect-Polymorphic Function Composition**).* We can also express effect-polymorphic function composition:

```
def >>(f: a -> b \ ef1, g: b -> c \ ef2) : a -> c \ {ef1, ef2} = ...
                                                  // ^^^^^^^^ effect union
```

Here the signature of >> (pronounced "forward function composition") expresses that if f is a function from a to b with effect ef_1 and g is a function from b to c with effect ef_2 then we can compose f and g to get a function from a to c whose effect is the union of the effects ef_1 and ef_2.

2.3 Programming with Effect Exclusion

In Section 2.1, we saw several examples of higher-order functions that were effect polymorphic but required the exclusion of certain specific effects. We can statically enforce such requirements in our extension of Flix using a new programming construct

$$e \text{ without } F$$

which specifies that when evaluating e it is forbidden to perform the F effect. In other words, e may have any effect *except* for the F effect. Furthermore, the type and effect system allows us to write programs using the without construct while supporting complete type and effect inference. We now give several examples, inspired by our case studies, of such uses.

*Example 2.6 (**Blocking the UI Thread**).* A common mistake in UI frameworks is to write an event listener that blocks the UI thread. This results in a user experience of a "hanging" application when a long-running event listener is invoked. We can use effect exclusion to ensure that event listeners do not block the UI thread. If we assume that every blocking operation is annotated with the Block effect then we can exclude this effect upon event listener registration:

For example, we can write the program:

```
def onMousePressed(listener) =
    def l(e) = listener(e) without {Block};
    ... // register listener l;

onMousePressed(event -> do println("Clicked!"))    // OK
onMousePressed(event -> do println(do readLine())) // NOT OK, read may block.
```

where the onMousePressed function permits the mouse event listener to have any effect *except* the Block effect. This ensures that listeners cannot block the UI thread. We infer the following type for onMousePressed:

```
def onMousePressed(listener: MouseEvent -> Unit \ {ef - Block})
```

where onMousePressed is effect polymorphic in ef which can be instantiated in any way, except that we always "subtract" the Block effect. Hence, no matter how we choose ef we can never pass a function to onMousePressed that has the Block effect.

*Example 2.7 (**Recursive Event Listener Registration**).* A common pitfall or code smell, when using event-based frameworks, are event listeners that themselves register (other) listeners. We can use the type and effect system to disallow such behavior:

```
def onMouseClick(f: Event -> Unit \ {ef - Register}): Unit \ { Register }
```

Here the onMouseClick function registers an event listener f which can have any effect *except* the Register effect. Since onMouseClick itself has the Register effect, we have ensured that mouse listeners cannot register other listeners.

*Example 2.8 (**Error Handlers**).* We can use the type and effect system to ensure that error handlers *themselves* cannot throw exceptions. For example:

```
def recoverWith(f: Unit -> a \ {ef, Throw}, h: Error -> a \ { ef - Throw })
```

Here the signature of recoverWith ensures that the handler function h cannot throw an exception, but may have any other effect.

*Example 2.9 (**A Safe Logger**).* We finish with an example inspired by a recent vulnerability[2] in Log4j2 whereby an attacker could gain access to a machine by sending a carefully crafted string which, when logged, was then inadvertently interpreted as a JNDI request leading to a malicious class being downloaded from the internet and loaded into the running JVM. Specifically, given a logger function, we can introduce a safeLogger function to ensure that the underlying logger cannot access the network nor load bytecode into the JVM:

```
def safeLogger(logger) = s -> logger(s) without {Network, ClassLoad}
```

Here the type and effect system will infer that the *original* logger function passed to safeLogger may have any effect *except* for the Network and ClassLoad effects. This example illustrates how the without construct together with our type and effect system can aid (and partially automate) security audits, an otherwise tedious and error-prone manual process.

3 CALCULUS

We now present $\lambda_{\complement}$, a calculus with a type and effect system [Nielson et al. 1999] that supports effect polymorphism and effect exclusion. The interesting term-level feature is the new without construct which forbids the execution of an effect. In this section, we start by presenting the syntax. In Section 3.2 we then present the operational semantics, in Section 3.3 we present the type system, and in Section 3.4 we connect the two by proving the usual progress and preservation theorems [Wright and Felleisen 1994] and a non-standard effect safety theorem, which intuitively states that well-typed programs never perform an effect that was forbidden by the without construct.

3.1 Syntax

Figure 1 defines the syntax of terms of $\lambda_{\complement}$, which includes variables (x), constants (c), function abstractions ($\lambda x.\, e$), and function applications ($v\, v$). It includes two different binding constructs: let $x = e_1$ in e_2 and def $x = v$ in e. The former sequences two potentially effectful expressions and binds the result of evaluating e_1 to x. The latter binds the value v to the variable x in e. As we

[2] https://nvd.nist.gov/vuln/detail/CVE-2021-44228

$$
\begin{array}{rcl}
v \in \mathit{Val} & = & x \mid c \mid \lambda x.\, e \\
e \in \mathit{Exp} & = & v \\
& \mid & v\, v \\
& \mid & \mathsf{let}\, x = e\, \mathsf{in}\, e \\
& \mid & \mathsf{def}\, x = v\, \mathsf{in}\, e \\
& \mid & \mathsf{if}\, v\, \mathsf{then}\, e\, \mathsf{else}\, e \\
& \mid & \mathsf{do}\, F(v) \\
& \mid & e\, \mathsf{without}\, F
\end{array}
$$

$$
\begin{array}{rcl}
x \in \mathit{Var} & = & \text{a set of variable symbols} \\
F \in \mathit{Eff} & = & \text{a set of effect operations} \\
c \in \mathit{Cst} & = & \text{a set of primitive constants}
\end{array}
$$

Abbreviations.

$$
\begin{array}{rcl}
e_1\, e_2 & \doteq & \mathsf{let}\, x_1 = e_1\, \mathsf{in}\, \mathsf{let}\, x_2 = e_2\, \mathsf{in}\, x_1\, x_2 \\
\mathsf{if}\, e_1\, \mathsf{then}\, e_2\, \mathsf{else}\, e_3 & \doteq & \mathsf{let}\, x_1 = e_1\, \mathsf{in}\, \mathsf{if}\, x_1\, \mathsf{then}\, e_2\, \mathsf{else}\, e_3 \\
\mathsf{do}\, F(e) & \doteq & \mathsf{let}\, x = e\, \mathsf{in}\, \mathsf{do}\, F(x) \\
e_1\, ;\, e_2 & \doteq & \mathsf{let}\, _ = e_1\, \mathsf{in}\, e_2
\end{array}
$$

Fig. 1. Syntax of terms in λ_{C}.

will see in Section 3.3, we only generalize the type of def-bound values, which enforces a value restriction [Kammar and Pretnar 2017]. We also include an if-then-else expression to demonstrate how the type and effect system handles control-flow merges.

Importantly, the calculus includes two constructs for working with effects: First, the $\mathsf{do}\, F(v)$ expression performs the effect operation F with argument v. Second, the e without F expression ensures that evaluating the potentially effectful expression e cannot have effect F. Both effect operations and effects are drawn from the same set of external effects denoted by *Eff*, e.g. Block, Register, or Throw.

To streamline the presentation of the operational semantics, and to simplify our proofs, we present the calculus in fine-grain call-by-value [Levy et al. 2003]. In this style, every non-trivial subexpression needs to be let-bound. This is also the reason why variables are included in the syntactic category of values. As defined in Figure 1, the more familiar form of function application can be recovered as syntactic sugar:

$$e_1\, e_2 \doteq \mathsf{let}\, x_1 = e_1\, \mathsf{in}\, \mathsf{let}\, x_2 = e_2\, \mathsf{in}\, x_1\, x_2$$

The abbreviation makes explicit that we first evaluate the two potentially effectful expressions in sequence and only then apply the resulting function to the argument value. The other forms can be recovered similarly.

3.2 Semantics

Figure 2 defines the semantics of λ_{C} as an abstract machine. We use fine-grain call-by-value to avoid congruence rules entirely and focus on the interesting cases only. A machine state (Subfigure 2a) is a pair $\langle\, e \mid k \,\rangle$ consisting of an expression e and a call stack k. A stack is a list of frames, where each frame is either a let-frame or a without-frame. While the let-frames are a standard way of modeling the current evaluation context, the without-frames are special to this abstract machine. They allow the machine to detect when it is about to perform a forbidden effect. The initial state of the machine is $\langle\, e \mid \bullet \,\rangle$, where e is a closed expression and $\bullet$ denotes the empty stack.

The reduction steps fall into three groups: (1) steps that do not modify the stack, (2) steps that push frames onto the stack, and (3) steps that pop frames from the stack.

Group (1) includes standard reduction steps for definitions, function application, and conditionals. The interesting step in this group is the last step for do-expressions. In this formalization, it ignores its argument and returns the unit value. Of course, in a real system, it would have actual side effects, for example getting the current time, opening a file, reading bytes from the network, and so on.

$$
\begin{array}{rcl}
f \in \mathit{Frame} & = & \mathsf{let}\, x = \square\, \mathsf{in}\, e \\
& | & \square\, \mathsf{without}\, F \\
k \in \mathit{Stack} & = & f :: k \\
& | & \bullet \\
m \in \mathit{Machine} & = & \langle\, e \mid k \,\rangle
\end{array}
$$

(a) Machine states of $\lambda_{\complement}$.

$$
\begin{array}{l}
\mathsf{forbidden}(\mathsf{let}\, x = \square\, \mathsf{in}\, e :: k) \\
\quad = \mathsf{forbidden}(k) \\
\mathsf{forbidden}(\square\, \mathsf{without}\, F :: k) \\
\quad = \mathsf{forbidden}(k) \cup \{F\} \\
\mathsf{forbidden}(\bullet) \\
\quad = \varnothing
\end{array}
$$

(b) Predicate stating which effects are forbidden by a context.

Group 1: Reduction steps that do not modify the stack.

$$
\begin{array}{rcl}
\langle\, \mathsf{def}\, x = v\, \mathsf{in}\, e \mid k \,\rangle & \longrightarrow & \langle\, e[x \mapsto v] \mid k \,\rangle \\
\langle\, (\lambda x.\, e)\, v \mid k \,\rangle & \longrightarrow & \langle\, e[x \mapsto v] \mid k \,\rangle \\
\langle\, \mathsf{if}\, \mathsf{true}\, \mathsf{then}\, e_1\, \mathsf{else}\, e_2 \mid k \,\rangle & \longrightarrow & \langle\, e_1 \mid k \,\rangle \\
\langle\, \mathsf{if}\, \mathsf{false}\, \mathsf{then}\, e_1\, \mathsf{else}\, e_2 \mid k \,\rangle & \longrightarrow & \langle\, e_2 \mid k \,\rangle \\
\langle\, \mathsf{do}\, F(v) \mid k \,\rangle,\ F \notin \mathsf{forbidden}(k) & \longrightarrow & \langle\, () \mid k \,\rangle
\end{array}
$$

Group 2: Reduction steps that push frames.

$$
\begin{array}{rcl}
\langle\, \mathsf{let}\, x = e_1\, \mathsf{in}\, e_2 \mid k \,\rangle & \longrightarrow & \langle\, e_1 \mid \mathsf{let}\, x = \square\, \mathsf{in}\, e_2 :: k \,\rangle \\
\langle\, e\, \mathsf{without}\, F \mid k \,\rangle & \longrightarrow & \langle\, e \mid \square\, \mathsf{without}\, F :: k \,\rangle
\end{array}
$$

Group 3: Reduction steps that pop frames.

$$
\begin{array}{rcl}
\langle\, v \mid \mathsf{let}\, x = \square\, \mathsf{in}\, e :: k \,\rangle & \longrightarrow & \langle\, e[x \mapsto v] \mid k \,\rangle \\
\langle\, v \mid \square\, \mathsf{without}\, F :: k \,\rangle & \longrightarrow & \langle\, v \mid k \,\rangle
\end{array}
$$

(c) Machine steps of $\lambda_{\complement}$ as an abstract machine.

Fig. 2. Abstract machine semantics of $\lambda_{\complement}$.

Importantly, the precondition on this step states that we can only perform an effect if the effect is not forbidden by the current stack. The meta function forbidden in Figure 2b defines which set of effects is forbidden by a given stack: precisely those that occur in without-frames.

Group (2) includes two reduction steps, which focus on the expression that should be evaluated next. They either push a let-frame or a without-frame onto the stack. A newly pushed without-frame forbids the effect F while the expression e is being evaluated.

Group (3) includes two reduction steps, for when the expression under evaluation is a value. The first is the typical let-binding rule for the call-by-value lambda calculus, substituting the value for the bound variable in the expression. The second rule models returning *values* through a without-expression. It pops the without-frame from the top of the stack.

Example 3.1. We can express a variant of Example 2.6 in $\lambda_{\complement}$ as follows:

$$
\begin{array}{l}
\mathsf{def}\ \mathit{onMousePressed} = \lambda\, \mathit{listener}.\, \mathit{listener}\,()\ \mathsf{without}\ \mathsf{Block}\ \mathsf{in} \\
\mathit{onMousePressed}\,(\lambda_.\ \ \mathsf{do}\ \mathsf{Print}(\text{“Clicked!”})\ )
\end{array}
$$

The above example only models the first call to the higher-order function *onMousePressed*, which uses the without construct to prevent Block effects. The argument function uses the Print effect,

$$
\begin{array}{rcl}
\varphi \in \mathit{Effect} & = & \beta \mid \varnothing \mid \{F\} \mid \varphi^c \mid \varphi \cup \varphi \mid \varphi \cap \varphi \\
\tau \in \mathit{Type} & = & \alpha \mid \iota \mid \tau \xrightarrow{\varphi} \tau \\
\sigma \in \mathit{Scheme} & = & \tau \mid \forall \alpha.\, \sigma \mid \forall \beta.\, \sigma
\end{array}
\qquad
\begin{array}{rcl}
\iota \in \mathit{BaseType} & = & \mathsf{Unit} \mid \mathsf{Bool} \mid \mathsf{String} \mid \cdots \\
\alpha \in \mathit{TypeVar} & = & \text{a set of type variables} \\
\beta \in \mathit{EffectVar} & = & \text{a set of effect variables}
\end{array}
$$

Fig. 3. Syntax of types in $\lambda_{\complement}$.

which we expect to evaluate without getting stuck. To verify this intuition and to illustrate the operational semantics, let us execute this example, after substituting the definition of *onMousePressed*.

$$
\begin{aligned}
&\langle\, (\lambda\, \mathit{listener}.\, \mathit{listener}\,()\ \mathsf{without}\ \mathsf{Block})\ (\lambda_.\ \mathsf{do}\ \mathsf{Print}(\text{“Clicked!”})) \mid \bullet \,\rangle \\
&\longrightarrow \langle\, (\lambda_.\ \mathsf{do}\ \mathsf{Print}(\text{“Clicked!”}))\ ()\ \mathsf{without}\ \mathsf{Block} \mid \bullet \,\rangle \\
&\longrightarrow \langle\, (\lambda_.\ \mathsf{do}\ \mathsf{Print}(\text{“Clicked!”}))\ () \mid \square\ \mathsf{without}\ \mathsf{Block}\ ::\ \bullet \,\rangle \\
&\longrightarrow \langle\, \mathsf{do}\ \mathsf{Print}(\text{“Clicked!”}) \mid \square\ \mathsf{without}\ \mathsf{Block}\ ::\ \bullet \,\rangle \\
&\longrightarrow \langle\, () \mid \square\ \mathsf{without}\ \mathsf{Block}\ ::\ \bullet \,\rangle \\
&\longrightarrow \langle\, () \mid \bullet \,\rangle
\end{aligned}
$$

Running the example performs the effect Print and evaluates to unit, as expected.

Example 3.2. Now consider what happens when we attempt to perform a Block effect.

$$
\begin{aligned}
&\mathsf{def}\ \mathit{onMousePressed} = \lambda\, \mathit{listener}.\, \mathit{listener}\,()\ \mathsf{without}\ \mathsf{Block}\ \mathsf{in} \\
&\mathit{onMousePressed}\,(\lambda_.\ \mathsf{do}\ \mathsf{Block}(())\ )
\end{aligned}
$$

In this second example, the without construct should prevent the *listener*'s attempt to block.

$$
\begin{aligned}
&\langle\, (\lambda\, \mathit{listener}.\, \mathit{listener}\,()\ \mathsf{without}\ \mathsf{Block})\ (\lambda_.\ \mathsf{do}\ \mathsf{Block}(())) \mid \bullet \,\rangle \\
&\longrightarrow \langle\, (\lambda_.\ \mathsf{do}\ \mathsf{Block}(()))\ ()\ \mathsf{without}\ \mathsf{Block} \mid \bullet \,\rangle \\
&\longrightarrow \langle\, (\lambda_.\ \mathsf{do}\ \mathsf{Block}(()))\ () \mid \square\ \mathsf{without}\ \mathsf{Block}\ ::\ \bullet \,\rangle \\
&\longrightarrow \langle\, \mathsf{do}\ \mathsf{Block}(()) \mid \square\ \mathsf{without}\ \mathsf{Block}\ ::\ \bullet \,\rangle
\end{aligned}
$$

Indeed, the machine gets stuck because $\mathsf{Block} \in \mathsf{forbidden}(\square\ \mathsf{without}\ \mathsf{Block}\ ::\ \bullet)$. Programmers deemed blocking to be undesirable and our operational semantics prevented it at runtime.

3.3 Type System

The semantics of the term-level construct without allows us to restrict the effects an expression can have at runtime. Instead of performing an undesirable effect the machine gets stuck. Now we present a type and effect system for $\lambda_{\complement}$ which allows us to rule out this kind of situation entirely, while still being expressive and supporting complete inference. As we will see in Section 3.4, the usual progress and preservation theorems ensure that well-typed programs never get stuck.

3.3.1 Syntax of Types. Figure 3 defines the syntax of types. Effects φ are *set formulas*. We use β to denote an effect variable, $\varnothing$ to denote the empty set of effects, $\{F\}$ to denote a singleton effect set, φ^c to denote the complement of φ, $\varphi_1 \cup \varphi_2$ to denote the union of the effect sets φ_1 and φ_2, and finally $\varphi_1 \cap \varphi_2$ to denote the intersection of the effect sets φ_1 and φ_2. Subtraction, i.e. $\varphi_1 - \varphi_2$, can be expressed as $\varphi_1 \cap \varphi_2^c$. Given a finite set of (user-defined) effects U, the language of effects is a Boolean algebra of sets where $\bot = \emptyset$ and $\top = U$ (the universe of effects). In addition to constant sets, an effect set formula φ may contain variables $\beta_1, \cdots, \beta_n$. We write $\mathsf{fvs}(\varphi)$ for the variables in φ. A *valuation* ν for a formula φ is an assignment of concrete sets of effects to all of the variables $\mathit{fvs}(\varphi)$.

An effect set formula is a function from valuations to a concrete set. Two effect set formulas φ_1 and φ_2 are *equivalent* (written $\varphi_1 \equiv_{\mathcal{B}} \varphi_2$) if they describe the same function. That is, if $\forall \nu.\ \nu(\varphi_1) = \nu(\varphi_2)$ where ν must be a valuation of both φ_1 and φ_2. We give a few examples of this equivalence, wherein all of them $U = \{A, B, C\}$:

Example 3.3. $\{A\} \cup \{B\} \equiv_{\mathcal{B}} \{C\}^c$.

Example 3.4. $(\{A\} \cup \{B\}) \cap \beta \equiv_{\mathcal{B}} \beta \cap \{A, B\}$.

Example 3.5. $\beta \cup \beta^c \equiv_{\mathcal{B}} \{A, B, C\}$.

Example 3.6. $\beta_1 \cup (\{A\} \cap (\{B\} \cap \beta_2)) \equiv_{\mathcal{B}} \beta_1 \cup (\{A\} \cap (\{C\} \cap \beta_3))$.

The last two examples show that φ_1 and φ_2 do not need to have the same variables to be equivalent.

We extend our notion of equivalence between sets of effects to types. We define $\tau \equiv_{\mathcal{B}} \tau'$ as the smallest relation such that:

- If $\tau = \tau'$ then $\tau \equiv_{\mathcal{B}} \tau'$.
- If $\tau_1 \equiv_{\mathcal{B}} \tau_1'$, $\tau_2 \equiv_{\mathcal{B}} \tau_2'$ and $\varphi \equiv_{\mathcal{B}} \varphi'$, then $\tau_1 \xrightarrow{\varphi} \tau_2 \equiv_{\mathcal{B}} \tau_1' \xrightarrow{\varphi'} \tau_2'$.

For example, we have that:

$$\tau \xrightarrow{\varnothing} (\tau \xrightarrow{\{F_1\} \cup \{F_2\}} \tau) \equiv_{\mathcal{B}} \tau \xrightarrow{\beta \cap \beta^c} (\tau \xrightarrow{\{F_1, F_2\}} \tau)$$

A key idea, which enables type and effect inference in our system, is that Boolean algebras have most-general unifiers. We can solve such unification problems on sets using the Successive Variable Elimination (SVE) algorithm.

Since our type system uses Hindley-Milner style polymorphism [Damas 1984; Hindley 1969; Milner 1978], types are split into mono types τ and type schemes σ. Mono types consist of type variables α, a set of base types denoted by ι (e.g. `Unit`, `Bool`, `String`, etc.), and function types $\tau_1 \xrightarrow{\varphi} \tau_2$, which represent functions from values of type τ_1 to values of type τ_2 with *latent* effects φ. That is, the effects φ are performed when the function is applied. Type schemes consist of mono types τ, and also of quantified types $\forall \alpha.\sigma$ where α is a type variable, and $\forall \beta.\sigma$ where β is an effect variable. A type scheme σ is of the form $\sigma = \forall \overline{\gamma}.\tau$ where $\overline{\gamma}$ is a sequence of type and effect variables.

Finally, we define type substitutions S as assignments of type variables to types and of effect variables to effects. We say the type τ is an instance of type scheme σ, written $\sigma \sqsubseteq \tau$, if $\sigma = \forall \overline{\gamma}.\tau'$ and there exists a type substitution S such that $\mathrm{dom}(S) = \overline{\gamma}$ and $S(\tau') = \tau$. Moreover, we define a context Γ as a set of bindings $x : \sigma$, and $\mathrm{ftv}(\sigma)$ to be the type variables that occur free in σ, and $\mathrm{ftv}(\Gamma)$ as the union of all free type variables in its range. We also define the generalization of a type $\mathrm{gen}(\Gamma, \tau)$ as $\forall \alpha_1, \cdots, \forall \alpha_n. \forall \beta_1, \cdots, \forall \beta_n. \tau$ where $\{\alpha_1, \cdots, \alpha_n, \beta_1, \cdots, \beta_n\} = \mathrm{ftv}(\tau) \setminus \mathrm{ftv}(\Gamma)$.

3.3.2 Typing Rules. Figure 4 defines the typing rules of $\lambda_{\subseteq}$. The type judgment for expressions is of the form $\Gamma \vdash_{\mathrm{e}} e : \tau \mid \varphi$ and states that under typing environment Γ, the expression e has type τ and effects φ. We now discuss each rule in detail. Except for (T-Eq), all typing rules are syntax-directed.

The (T-Eq) rule states that if an expression e has type τ_1 and effects φ_1 and if τ_1 is equivalent to τ_2 and φ_1 is equivalent to φ_2, then we may conclude that e has type τ_2 and effects φ_2. This rule embodies the notion of types modulo Boolean equivalence.

The (T-Var) rule states that if the variable x has the type scheme σ then we can type x as any instance of the type scheme, i.e. any type where $\sigma \sqsubseteq \tau$. A variable has empty effects $\varnothing$.

The (T-Cst) rule states that a constant c has the type assigned to it by a typeOf function. We assume a set of base types (e.g. Unit, Bool, String) and that the typeOf function assigns a base type to each constant. A constant has empty effects $\varnothing$.

$\boxed{\Gamma \vdash_{\mathrm{e}} e : \tau \mid \varphi}$

$$\frac{\Gamma \vdash_{\mathrm{e}} e : \tau_1 \mid \varphi_1 \quad \tau_1 \equiv_{\mathcal{B}} \tau_2 \quad \varphi_1 \equiv_{\mathcal{B}} \varphi_2}{\Gamma \vdash_{\mathrm{e}} e : \tau_2 \mid \varphi_2}\ (\text{T-Eq})$$

$$\frac{\Gamma \vdash_{\mathrm{e}} v_1 : \tau_1 \mid \varnothing \quad \Gamma, x : \sigma \vdash_{\mathrm{e}} e_2 : \tau_2 \mid \varphi \quad \sigma = \mathrm{gen}(\Gamma, \tau_1)}{\Gamma \vdash_{\mathrm{e}} \mathrm{def}\, x = v_1 \,\mathrm{in}\, e_2 : \tau_2 \mid \varphi}\ (\text{T-Def})$$

$$\frac{(x, \sigma) \in \Gamma \quad \sigma \sqsubseteq \tau}{\Gamma \vdash_{\mathrm{e}} x : \tau \mid \varnothing}\ (\text{T-Var})$$

$$\frac{\Gamma \vdash_{\mathrm{e}} e_1 : \tau_1 \mid \varphi_1 \quad \Gamma, x : \tau_1 \vdash_{\mathrm{e}} e_2 : \tau_2 \mid \varphi_2}{\Gamma \vdash_{\mathrm{e}} \mathrm{let}\, x = e_1 \,\mathrm{in}\, e_2 : \tau_2 \mid \varphi_1 \cup \varphi_2}\ (\text{T-Let})$$

$$\frac{\mathrm{typeOf}(c) = \iota}{\Gamma \vdash_{\mathrm{e}} c : \iota \mid \varnothing}\ (\text{T-Cst})$$

$$\frac{\Gamma \vdash_{\mathrm{e}} v_1 : \mathsf{Bool} \mid \varnothing \quad \Gamma \vdash_{\mathrm{e}} e_2 : \tau \mid \varphi_2 \quad \Gamma \vdash_{\mathrm{e}} e_3 : \tau \mid \varphi_3}{\Gamma \vdash_{\mathrm{e}} \mathrm{if}\, v_1 \,\mathrm{then}\, e_2 \,\mathrm{else}\, e_3 : \tau \mid \varphi_2 \cup \varphi_3}\ (\text{T-Ite})$$

$$\frac{\Gamma, x : \tau_1 \vdash_{\mathrm{e}} e : \tau_2 \mid \varphi}{\Gamma \vdash_{\mathrm{e}} \lambda x.\, e : \tau_1 \xrightarrow{\varphi} \tau_2 \mid \varnothing}\ (\text{T-Abs})$$

$$\frac{\Gamma \vdash_{\mathrm{e}} v : \tau \mid \varnothing \quad \mathrm{signatureOf}(F) = \tau \to \mathsf{Unit}}{\Gamma \vdash_{\mathrm{e}} \mathrm{do}\, F(v) : \mathsf{Unit} \mid \{F\}}\ (\text{T-Do})$$

$$\frac{\Gamma \vdash_{\mathrm{e}} v_1 : \tau_1 \xrightarrow{\varphi} \tau_2 \mid \varnothing \quad \Gamma \vdash_{\mathrm{e}} v_2 : \tau_1 \mid \varnothing}{\Gamma \vdash_{\mathrm{e}} v_1\, v_2 : \tau_2 \mid \varphi}\ (\text{T-App})$$

$$\frac{\Gamma \vdash_{\mathrm{e}} e : \tau \mid \varphi \quad \varphi \cap \{F\} = \varnothing}{\Gamma \vdash_{\mathrm{e}} e \,\mathrm{without}\, F : \tau \mid \varphi}\ (\text{T-Without})$$

Fig. 4. Typing rules of $\lambda_{\complement}$.

The (T-Abs) and (T-App) rules type lambda abstractions and applications. An abstraction takes the effects of an expression e and moves it onto the arrow type whereas an application releases the latent effects on an arrow type. The more traditional rule for application follows directly from the definition of abbreviations in Figure 1 and applications of rules (T-Let) and (T-App).

$$\frac{\Gamma \vdash_{\mathrm{e}} e_1 : \tau_1 \xrightarrow{\varphi} \tau_2 \mid \varphi_1 \quad \Gamma \vdash_{\mathrm{e}} e_2 : \tau_1 \mid \varphi_2}{\Gamma \vdash_{\mathrm{e}} e_1\, e_2 : \tau_2 \mid \varphi_1 \cup \varphi_2 \cup \varphi}\ (\text{T-App-Eff})$$

The (T-Def) rule types the definition of a name for a value. The type of the bound variable x is generalized from τ_1 to σ. As usual in languages with effects, we only generalize the binding of values [Wright 1995]. We believe this restriction could be relaxed to a purity restriction [Kammar and Pretnar 2017].

The (T-Let) rule types the sequencing of two potentially effectful expressions. The result of evaluating e_1 is bound to the variable x. No generalization happens in this case. Moreover, the effect of the compound expression aggregates the effects of both subexpressions $\varphi_1 \cup \varphi_2$.

The (T-Ite) rule types if-then-else expressions. As is standard, the rule requires that the conditional expression has type Bool and that both branches have the same type τ. The effects of the if-then-else expression is the union of the effects of the two branches.

The (T-Do) rule states that we can invoke an external effect F if its argument has the expected type according to a signatureOf function which assigns types to effect operations. A do expression has the singleton effect $\{F\}$. In this formalization, effect operations must always return the value of type `Unit`.

The (T-Without) rule states that we can type an expression e without an effect F, if the effects φ of e do not contain F. We express this non-membership by requiring the intersection of φ with F to be empty.

Example 3.7. We can enforce that the composition of two functions f and g lack the Console effect, i.e. *neither* f nor g can have the Console effect:

$$\mathsf{def}\, c_1 = \lambda f.\, \lambda g.\, \lambda x.\ g\,(f\,x)\ \mathsf{without\ Console}$$

Here c_1 has the type:

$$(a \xrightarrow{e_1 \setminus \{\mathsf{Console}\}} b) \to (b \xrightarrow{e_2 \setminus \{\mathsf{Console}\}} c) \to (a \xrightarrow{(e_1 \cup e_2) \setminus \{\mathsf{Console}\}} c)$$

where f and g are free to have any effects except Console, and the result of the composition has the union of their effects, without the Console effect. In our type and effect system, expressions have principal types only up to Boolean equivalence. For example, the c_1 function can also be given the syntactically different but equivalent type:

$$(a \xrightarrow{e_1 \cap (e_2 \setminus \{\mathsf{Console}\})} b) \to (b \xrightarrow{e_1 \cap ((e_3 \cup \overline{e_2}) \setminus \{\mathsf{Console}\})} c) \to (a \xrightarrow{e_1 \setminus \{\mathsf{Console}\}} c)$$

Example 3.8. Continuing our function composition example, we can also enforce that *only* f lack the Console effect:

$$\mathsf{def}\, c_2 = \lambda f.\, \lambda g.\, \lambda x.\, g\ (f\,x\ \mathsf{without\ Console})$$

Here c_2 has the type:

$$(a \xrightarrow{e_1 \setminus \{\mathsf{Console}\}} b) \to (b \xrightarrow{e_2} c) \to (a \xrightarrow{(e_1 \setminus \{\mathsf{Console}\}) \cup e_2})$$

where f's effects must not include Console, while g may have any effects. The resulting function has the union of f's and g's effects.

Example 3.9. In the last example, Console and Throw are disallowed for f, and Throw for g.

$$\mathsf{def}\, c_3 = \lambda f.\, \lambda g.\, \lambda x.\, g\,(f\,x\ \mathsf{without\ Console})\ \mathsf{without\ Throw}$$

Here c_3 has the type:

$$(a \xrightarrow{e_1 \setminus \{\mathsf{Console,Throw}\}} b) \to (b \xrightarrow{e_2 \setminus \{\mathsf{Throw}\}} c) \to (a \xrightarrow{((e_1 \setminus \{\mathsf{Console}\}) \cup e_2) \setminus \{\mathsf{Throw}\}} c)$$

As these examples illustrate, by choosing where to place the without construct, programmers have fine-grained control over where effects are permitted and where they are forbidden.

3.4 Theorems

In Section 3.2, we have seen the semantics of $\lambda_{\complement}$ as an abstract machine. The machine is designed to get stuck whenever an effect operation is used in a context where it is forbidden. In Section 3.3, we have seen the type and effect system of $\lambda_{\complement}$, which tracks the effects an expression has but also which effects it must not have. Now we connect the two and prove that well-typed programs never get stuck. More specifically, our main result (Theorem 3.14) is that a well-typed program will never attempt to use a forbidden effect at runtime.

To this end, Figure 5 states invariants that are maintained throughout reduction. The typing judgment for machines $\vdash_{\mathsf{m}} \langle\, e \mid k \,\rangle$ **ok** defines what it means for a machine state to be well-typed, connecting the typing of the expression e with the typing of the stack k. The closed expression e has type τ and effects φ_1. Intuitively, the stack k accepts values of type τ and also *forbids* effects φ_2. To guarantee that no used effects (φ_1) are forbidden (φ_2), the effects φ_1 and φ_2 must be disjoint, i.e.

$\boxed{\vdash_{\mathsf{m}} m\ \mathbf{ok}}$

$$\frac{\vdash_{\mathsf{e}} e : \tau \mid \varphi_1 \quad \tau \vdash_{\mathsf{k}} k \setminus \varphi_2 \quad \varphi_1 \cap \varphi_2 \equiv_{\mathcal{B}} \varnothing}{\vdash_{\mathsf{m}} \langle\, e \mid k \,\rangle\ \mathbf{ok}} \text{ (T-Machine)}$$

$\boxed{\tau \vdash_{\mathsf{k}} k \setminus \varphi}$

$$\frac{}{\tau \vdash_{\mathsf{k}} \bullet \setminus \varnothing} \text{ (T-Done)} \qquad \frac{\tau \vdash_{\mathsf{k}} k \setminus \varphi}{\tau \vdash_{\mathsf{k}} \square\,\mathsf{without}\,F :: k \setminus \varphi \cup \{F\}} \text{ (T-Forbid)}$$

$$\frac{x : \tau \vdash_{\mathsf{e}} e : \tau_1 \mid \varphi_1 \quad \varphi_1 \cap \varphi_2 \equiv_{\mathcal{B}} \varnothing \quad \tau_1 \vdash_{\mathsf{k}} k \setminus \varphi_2}{\tau \vdash_{\mathsf{k}} \mathsf{let}\, x = \square\,\mathsf{in}\, e :: k \setminus \varphi_2} \text{ (T-Push)}$$

Fig. 5. Machine- and stack typing rules to maintain invariants for $\lambda_{\complement}$.

their intersection must be empty. As we will see shortly, these invariants are enough to prove our main effect safety theorem while at the same being maintained throughout reduction.

More concretely, the typing judgement for stacks $\tau \vdash_{\mathsf{k}} k \setminus \varphi$ types stacks k. The empty stack $\bullet$ accepts a value of any type and forbids no effects. A without-frame ($\square\,\mathsf{without}\,F$) accepts the same type as the rest of the stack and forbids the effect F in addition to the effects forbidden by the rest of the stack. A let-frame ($\mathsf{let}\, x = \square\,\mathsf{in}\, e$) accepts type τ when the expression e is well-typed given the variable x has type τ. The effects of e must not be forbidden by the rest of the stack k, because the expression will be evaluated in this context. It forbids the same effects as the rest of the stack k. Moreover, the type of the expression e and the type the stack k accepts have to agree, because eventually a value of this type will be returned to the rest of the stack.

Given these invariants, we prove progress and preservation theorems on machine states. By defining the operational semantics of $\lambda_{\complement}$ as an abstract machine with a stack, and by assigning types to stacks and machines, we can formalize and prove properties of the interaction between the program and its context. The full proofs are provided as supplementary material.

Theorem 3.10 (Progress). *If $\vdash_m m$ **ok**, then either there is a value v such that $m = \langle\, v \mid \bullet \,\rangle$, or there is a machine m' such that $m \longrightarrow m'$.*

Proof. By induction on the typing derivation. □

Theorem 3.11 (Preservation). *If $\vdash_m m$ **ok** and $m \longrightarrow m'$, then $\vdash_m m'$ **ok**.*

Proof. By case analysis on the stepping relation. □

The proof of progress uses the invariants described above, while the proof of preservation shows that they are maintained throughout reduction. In the proof of progress (case T-Do), we use our key effect safety theorem (Theorem 3.14), to show that indeed in a well-typed machine state we are never about to perform an effect which is forbidden. Its proof uses two trivial lemmas that relate expressions and stacks with the effects in their judgments respectively.

Lemma 3.12 (Performed effect). *If $\vdash_e do\, F(v) : \tau \mid \varphi_1$, then $\varphi_1 \equiv_{\mathcal{B}} \{F\}$.*

Lemma 3.13 (Forbidden effects). *If $\tau \vdash_k k \setminus \varphi_2$, then $\varphi_2 = forbidden(k)$.*

Since contexts can disallow the use of certain effects, our key effect safety theorem is non-standard. It states that well-typed machine states only perform effects that are not forbidden by the stack.

Theorem 3.14 (Effect safety). *If* $\vdash_m \langle\, do\, F(v) \mid k \,\rangle$ ***ok****, then* $F \notin forbidden(k)$.

Proof. We know that the machine state is well-typed and inversion on its typing relation gives us: **(1)** $\vdash_e do\, F(v) : \tau \mid \varphi_1$, and **(2)** $\tau \vdash_k k \setminus \varphi_2$, and **(3)** $\varphi_1 \cap \varphi_2 = \emptyset$. By using Lemma 3.12 on (1), we learn that **(4)** $\varphi_1 \equiv_{\mathcal{B}} \{F\}$. By using Lemma 3.13 on (2) we learn that **(5)** $\varphi_2 = \text{forbidden}(k)$. By elementary reasoning on sets with (3), (4), and (5), we conclude $F \notin \text{forbidden}(k)$. □

We close this section with a discussion of two limitations of the current system: the lack of sub-effecting (Section 3.5) and the closed effect universe assumption (Section 3.6).

3.5 Sub-Effecting

The proposed type and effect system lacks sub-effecting. Consequently, the program

```
if (true) (x -> do println(x)) else (x -> ())
```

is rejected by the type system. The two lambda expressions are respectively typed

$$\text{String} \xrightarrow{\text{Print}} \text{Unit} \qquad \text{and} \qquad \alpha \xrightarrow{\emptyset} \text{Unit}$$

which cannot unify—not even modulo Boolean equivalence—because one function has an effect and the other is pure. If the system supported sub-effecting, it would be possible to widen the type of the *else* branch, i.e., to type the pure function *as if* it had the Print effect, and hence the entire expression would be well-typed. However, despite the lack of sub-effecting, in some cases we can rewrite the program such that it is well-typed. For example, here we can push the if-then-else into the body of the lambda abstraction:

```
x -> (if (true) do println(x) else ())
```

The two sub-expressions of the if-then-else can be typed as Unit ! {Print} and Unit ! ∅. With the (T-Ite) rule, the entire lambda expression is now well-typed. We could imagine extending the type and effect system with sub-effecting, but then we would most likely lose type inference.

3.6 Closed Effect Universe

The calculus makes the reasonable assumption that the effect universe is known and fixed, since we can only run closed and complete programs. Does this assumption break separate compilation? Consider Modules 1 and 2.

Module 1

```
eff A { def a(): Unit }
eff B { def b(): Unit }
eff C { def c(): Unit }

def f(): Unit \ {~C} =
  do A.a();
  do B.b()
```

Module 2

```
eff D { def d(): Unit }

def g(): Unit \ {D} =
  do D.d()

def h(): Unit \ {D} =
  do D.d()
```

Each of these modules is well-typed in isolation. In particular, the function f is well-typed because the universe of effects is $\{A, B, C\}$, so $\{C\}^{\complement} = \{A, B\}$. However, the composition of these two programs is no longer well-typed: Module 2 adds effect D to the universe, and the complement

of C changes as a result. We seem to have lost separate compilation. To overcome this issue, our implementation (Section 4) diverges from the calculus in the following important way: We treat the effect universe as infinite by modeling it as a symbolic constant during type inference. Thus any concrete set of effects is never equal to the universe. Hence, if we try to compile the function f, we actually get the error:

```
The declared type does not match the inferred type:

  Declared: Unit -> Unit \ ~C
  Inferred: Unit -> Unit \ {A, B}
```

In essence, programmers cannot depend on the assumption that the effect universe is finite. This detail can alternatively be understood as the idea that the program always contains one "extra" effect that the programmer cannot reference. This makes the type-and-effect system compositional in our implementation, but it does mean that more programs are (potentially) rejected.

We remark that programs like the above, where the programmer has used $\{C\}^{\complement}$ as a shorthand for $\{A, B\}$, are *not* the way we intend our system to be used; hence the implementation rejects it. In our system, programmers should not have to write types (because they are inferred), and if they do, complement should typically be of the form $\mathsf{ef} - \{E\}$ for some effect variable ef and some concrete effect E which expresses an effect exclusion.

As an alternative to the symbolic universe, sub-effecting would allow a programmer to write a function like f because even if the universe expands, it is still the case that $\{A, B\} \subseteq \{C\}^{\complement}$, hence the function would be well-typed. We leave this as interesting future work to explore.

4 IMPLEMENTATION

We have implemented the proposed type and effect system in the Flix programming language.

4.1 The Flix Programming Language

Flix[3] is a functional-first, imperative, and logic programming language that supports algebraic data types, pattern matching, higher-order functions, parametric polymorphism, type classes, higher-kinded types, polymorphic effects, first-class Datalog constraints, channel and process-based concurrency, and tail call elimination. Flix compiles to JVM bytecode and runs on the Java Virtual Machine. Flix comes with online documentation (https://doc.flix.dev), API documentation (https://api.flix.dev), and a fully-featured VSCode extension.

4.2 Type and Effect System Extension

Following the $\lambda_{\complement}$ calculus, we have extended Flix with the "without" construct and the type and effect system with sets of effects. In particular, programmers can now declare their own effects (e.g. Throw), refer to them with "do", and exclude them with "without". The Flix compiler is whole-program (i.e. requires all source code to be available at compilation time) due to its use of monomorphization and whole-program optimizations (e.g. cross-module inlining). This requirement fits well with the assumption in the calculus that we know the universe of effects.

The Flix compiler project is 200,000 lines of code, including the standard library and tests. Our extension required approximately 10,000 new or changed lines of code.
Our Flix extension is open source, freely available, and ready for use at:

flix.dev

[3] https://flix.dev/ and https://github.com/flix/flix/

4.3 Visual Studio Code Support

We have extended the Visual Studio Code support in Flix. Figure 6 shows a screenshot of the editor in action. Note that the program illustrates both functions that are effect-annotated and functions where the effects are fully inferred.

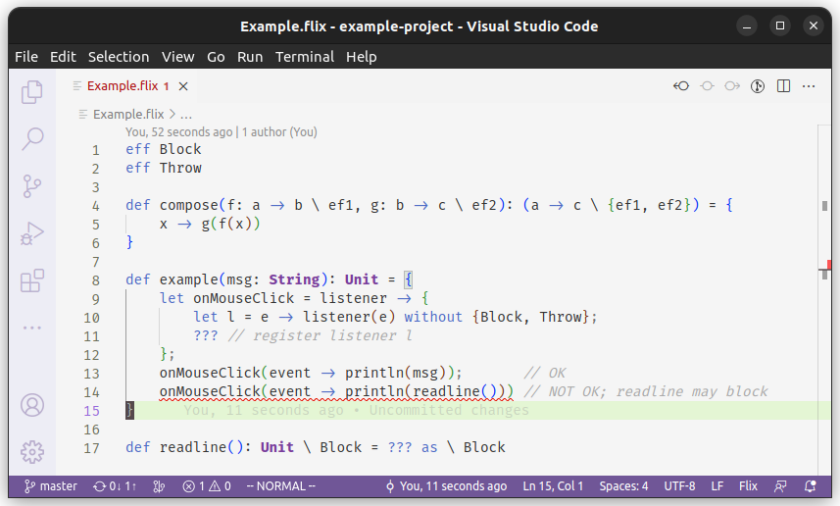


Fig. 6. A screenshot of our Visual Studio Code extension with complement effects.

4.4 Type Inference

The declarative type system presented in Section 3.3 supports type inference by a suitable extension of Algorithm W. The key observation is that Algorithm W solves unification problems on types and relies on the existence of most-general unifiers to compactly represent their solutions. The (T-Eq) rule considers two types to be equal *modulo Boolean equivalence* on their effect set formulas. In essence, we obtain the type inference rules from the declarative typing rules by replacing every "equality check" by a "unification check" which uses the SVE algorithm to compute most-general unifiers of the effect set formulas. We briefly mention three important implementation details: (i) Boolean unifiers may become very large and have to be simplified during unification, (ii) during let-generalization we must only consider "essential variables". For example, in the effect set formula: $(\beta_1 \cap \beta_1^C) \cup \beta_2$ the variable β_1 is non-essential whereas β_2 is essential, and (iii) for efficiency, inside the SVE algorithm, the top element (i.e. the effect universe) should be represented symbolically. We refer to [Madsen and van de Pol 2020] and [Martin and Nipkow 1989] for more details.

4.5 Additional Expressive Power

We have focused on the without construct and how it can be used to support effect exclusion. However, if we are willing to write explicit type and effect signatures, the proposed type and effect system is even more expressive and allows us to enforce invariants that cannot readily be expressed with the without construct alone. Note that our implementation allows us to write *some* type and effect signatures and have the rest inferred. We already saw one example of this in Section 2. We now give two more examples:

*Example 4.1 (**Upper Bounds**).* The type and effect system does not directly support sub-effecting. For example, a function with the single effect {Console} cannot be used where a function with

effect {Console, Throw} is expected, even though such use would be sound. But we *can* express upper bounds on the effects of an effect polymorphic function:

```
def hof(f: a -> Unit \ {(StdOut + StdErr) - ef})
```

Here the function parameter f may have any effect that is a subset of {StdOut, StdErr}, i.e., both effects, one of them, or neither of them. In other words, the function f is permitted to write standard out and standard error, but nothing else.

*Example 4.2 (**Non-Sharing**).* We can express that two functions may not share certain effects:

```
def hof(f: a -> a \ ef1 - (ef2 & Throw), g: a -> a \ ef2 - (ef1 & Throw))
```

Here f and g are permitted to have *any effects* but at most one of f and g can have the Throw effect. The idea is that f can have any effects ef_1 minus the effects of g (i.e. ef_2) upper bounded by Throw, and vise versa. For example, imagine that the effects of f are {Console, Throw}. Then $ef_1 = \{\text{Console}, \text{Throw}\}$ which means that the effect of g is: $ef_2 - (\{\text{Console}, \text{Throw}\} \cap \{\text{Throw}\})$ which is $ef_2 - \text{Throw}$. Thus g can have any effect, except Throw, as expected.

*Example 4.3 (**Disjoint Effects**).* We can also express that the effects of function parameters are disjoint. Given two functions f and g with effects ef_1 and ef_2, we might ask: when is it safe to execute the two functions in parallel? Clearly, if they read and write the same memory there is the potential for race conditions. However, if the effects of f and g are completely disjoint then it is safe to execute them in parallel. We can capture this in our type and effect system:

```
def par(x: a, f: a -> b \ ef1, g: a -> b \ {ef2 - ef1})
```

Here the par function takes a value x of type a and two functions f and g. The type signature of par enforces the effects of f and g to be disjoint. Specifically, if f has effects ef_1 then g may only have effects $ef_2 - ef_1$. In other words, any effect of f *cannot* be an effect of g.

This example again demonstrates that every function has multiple syntactically distinct principal types. In particular, we could have given the par function the equally most general type signature:

```
def par(x: a, f: a -> b \ {ef1 - ef2}, g: a -> b \ ef2)
```

In our type and effect system, expressions have principal types modulo Boolean equivalence of sets.

*Example 4.4 (**Disjoint Effects – Continued**).* In the previous example, we required that f and g should have completely disjoint effects. In practice, however, there are often effects that are safe to execute concurrently. For example, we can imagine that it would be safe for both f and g to use the network simultaneously. We might try to capture this situation with the signature:

```
def par(f: a -> b \ {ef1 + Network}, g: a -> b \ {(ef2 - ef1) + Network})
```

But this does not work as intended, since it requires both f and g to have the Network effect. We can see this because the effect sets of both f and g now include the Network effect and since our system does not have sub-effecting this would require f and g to have the network effect.

What we actually wanted to express is:

```
def par(x: a, f: a -> b \ ef1, g: a -> b \ (ef2 - (ef1 - Network)))
```

which allows us to instantiate ef_1 and ef_2 with effect sets that may contain the Network effect while still ensuring that the effects in ef_1 cannot overlap with those in ef_2.

5 EVALUATION

We conducted a case study of open-source projects identifying 59 program fragments that require effect exclusion for correctness. We demonstrate the usefulness of our type and effect system by recasting each of the fragments in our extension of Flix; i.e., we transform unenforced, informal documentation comments into Flix types and effects.

5.1 Methodology

We used GitHub's new code search tool (cs.github.com) to find documentation comments such as "must not raise", "must not throw", and "must not call". GitHub Search limits each query to one hundred results, hence to see more code, we refined each search to a particular programming language (e.g. by adding "language:scala" to the query).

We filtered these program fragments according to the criteria:

- The program fragment had to be an effect exclusion.
- The effect exclusion had to be unconditional[4].
- The program fragment had to be understandable.
- We limit ourselves to one program fragment per GitHub repository with at least 5 stars, and we excluded duplicates.

Observations. The program fragments fell largely into two categories: (1) functions that must not throw exceptions, and (2) functions that must not call certain other functions. Members of group (1) included object finalizers and destructors, functions intended to handle exceptions, and functions returning "Futures", whose exceptional status would presumably be contained within the returned object. Members of group (2) included functions whose arguments are in some intermediate state that must not be modified, functions with a lock on some object, and functions that must not call themselves to prevent infinite recursion.

5.2 Case Studies

We identified 59 program fragments in 15 different languages. For each program fragment, we were able to convert the unchecked documentation of effect exclusion into a statically-checked effect annotation (Table 1 lists an excerpt of these 59 fragments). In 5 cases, the recasting was not perfect: Either our Flix program was overly restrictive by forbidding an operation in general while the documentation only forbids it on a particular object, or the documentation included additional restrictions beyond effect exclusion. To provide further insight in our case study, in the remainder of this section, we present a selection of three example fragments and their translation to Flix.

[4]In other words, the effect exclusion should not depend on control– or data flow of the program. For example, we filtered out program fragments where an effect operation was excluded after another function had been called or where the exclusion of an effect operation depended on the application state.

#	Flix	Project	Stars	Description
1	✓	mldbai/mldb	628 ★	*The job MUST NOT throw an exception; any exception thrown within the job WILL CRASH THE PROGRAM*
2	✓	janestreet/async_unix	27 ★	*WARNING: Async code MUST NOT be used from within [f].*
3	(✓)	googleapis/google-cloud-go	3.0k ★	*f must not call Commit or Rollback on the provided Transaction.*
4	✓	JuliaIO/EzXML.jl	99 ★	*This function must not throw an exception; otherwise it may lead to a devastating tree*
5	✓	edenhill/librdkafka	6.1k ★	*The application MUST NOT call rd_kafka_destroy() from the event callback.*
6	✓	ogolem/ogolem	5 ★	*Must not throw Exceptions unless catastrophic condition exists.*
7	✓	RaghavSood/CompilingAndroidMail	18 ★	*MUST NOT throw any exceptions.*
8	✓	yo8192/fcron	87 ★	*we must NOT use xfclose_check()*
9	✓	neurolabs/henplus	95 ★	*Must not throw an Execption and may to whatever it needs to do.*
10	(✓)	scijava/scijava-common	67 ★	*It must be implemented highly thread safe and must not use any kind of locks*
11	✓	facebookincubator/antlir	38 ★	*IMPORTANT: This code must not write anything to stdout, the fd can be 1.*
12	✓	scottlamb/moonfire-nvr	691 ★	*The callback [...] must not call back into the database or block.*
13	✓	richfitz/thor	51 ★	*this must not throw: [...] Below here is OK though*
14	✓	graphile/persisted-operations	31 ★	*this function must not throw!*
15	✓	b0-system/b0	32 ★	*[k] must not raise.*
16	✓	dart-lang/sdk	8.3k ★	*Finalization callbacks must not throw.*
17	✓	re-motion/TypePipe	22 ★	*Implementations of this interface must not use other pipeline APIs themselves*
18	✓	decred/dcrd	648 ★	*This must not call any other connection manager methods during iteration or it will result in a deadlock [...]*
19	✓	arangodb/arangodb	12.5k ★	*This method must not throw under penalty of ...*
20	✓	janestreet/bonsai	214 ★	*This one must not throw*
21	✓	amule-project/amule	857 ★	*This function should not throw Either of the CSafeIOExceptions*
22	✓	uNetworking/uWebSockets	14.0k ★	*The drain callback must not publish, unsubscribe or subscribe.*
23	✓	ChainSafe/lodestar	657 ★	*MUST not throw an error in any case, related to cache miss.*
24	✓	ninedraft/huffy	7 ★	*Must not call t.Parallel()!*
25	✓	ocaml/dune	1.2k ★	*[handle_error_no_raise] must not raise exceptions [...] Nested calls of [run_with_error_handler] are not allowed.*
···	*33 additional case studies in the extended version* ···			
59	✓	pinojs/thread-stream	152 ★	*method should not throw an Error if something goes wrong but emit an error event*

Table 1. Case Studies: Each row represents a program fragment that relies on effect exclusion to ensure correctness. The columns are as follows: The Project and Stars columns show the name and number of stars of the GitHub repository, respectively. The Description column shows the comments that document the effect exclusion of the program fragment. Finally, the Flix column shows whether we were able to recast the program fragment in Flix using the implemented type and effect system.

*Case Study (**ocaml/dune**).* ocaml/dune is the principal build tool for OCaml programs. It is available on GitHub at https://github.com/ocaml/dune and has 1.2k stars. In the file `memo.mli`, the documentation of the function `run_with_error_handler` imposes two requirements: (1) The `handle_error_no_raise` parameter must not raise an exception, and (2) nested calls of the function are not allowed; i.e., the function passed to `handle_error_no_raise` must not itself call `handle_error_no_raise`. The consequences of failing to meet requirement (1) are broken invariants and "confusing errors"; and those of requirement (2) are unspecified.

```
val run_with_error_handler :
     (unit -> 'a t)
  -> handle_error_no_raise:(Exn_with_backtrace.t -> unit Fiber.t)
  -> 'a Fiber.t
```

With Flix, both of these requirements can be expressed through the type system. (1) can be addressed by allowing the `handle_error_no_raise` parameter to have any effect except raising an exception, giving it the type `ExnWithBacktrace -> Fiber[Unit] \ ef2 - Raise`. (2) can be addressed by creating an effect specific to the `runWithErrorHandler` function, indicating that this effect is used by the function, and allowing the `f` parameter to have any effect except this new `RunWithErrorHandler` effect, giving it the type `Unit -> T[a] \ ef1 - RunWithErrorHandler`.

```
eff RunWithErrorHandler
def runWithErrorHandler(
    f: Unit -> T[a] \ ef1 - RunWithErrorHandler,
    handleErrorNoRaise: ExnWithBacktrace -> Fiber[Unit] \ ef2 - Raise):
        Fiber[a] \ (ef1 + ef2 + RunWithErrorHandler) - Raise
```

*Case Study (**heavyai/heavydb**).* heavyai/heavydb is a parallelism-focused relation database engine. It is available on GitHub at https://github.com/heavyai/heavydb and has 2.7k stars. In the file `scope.h`, the inline documentation of the `ScopeGuard` destructor indicates that the function `at_exit_` must not throw an exception.

```
~ScopeGuard() {
  if (at_exit_) {
    // note that at_exit_ must not throw
    at_exit_();
  }
}
```

With Flix, the requirement can be expressed in the signature of the `at_exit_` function. The type `Unit -> Unit \ ef - Throw` allows any effect except the `Throw` effect.

```
enum ScopeGuard[ef: Eff]({
    atExit = Option[Unit -> Unit \ ef - Throw]
})
```

*Case Study (**amule-project/amule**).* amule-project/amule client for the ED2K file-sharing network. It is available on GitHub at https://github.com/amule-project/amule and has 857 stars. In the file `SafeFile.h`, the documentation of the `doRead` function indicates that it should not throw either of two particular `CSafeIOExceptions`.

```
virtual sint64 doRead(void* buffer, size_t count) const = 0;
```

With Flix, the requirement can be expressed in the type of the `doRead` function by introducing a `CSafeIOException`. The function `doRead`, represented by a record field in Flix, is permitted to have any effect *except* using a `CSafeIOException`.

```
eff CSafeIOException

enum CFileDataIO[ef: Eff, a: Type]({
    doRead = (a, SizeT) -> Int64 \ ef - CSafeIOException
})
```

6 TWO LARGER EXAMPLES

The case studies demonstrate a need for effect exclusion and show that our system can conceptually capture the required types and effects. Since the recast program fragments from the case study are not complete programs, we have also implemented two libraries that use effect exclusion together with applications that exercise them. The first program is a small GUI library, and the second is an event bus library.

6.1 GUI Library

The first example is a GUI-building library that wraps part of the Java Swing framework. It allows programmers to build a simple user interface with labels, buttons, and text fields. A typical expectation of a GUI is that it should be responsive: a slow computation should not generally prevent accessing or closing the rest of the interface. Our extension to Flix allows this property to expressed in the type system. The `addActionListener` function makes use of the `without` expression, requiring that the listener must not block; i.e., the listener's effect must not include the `Block` effect.

```
def addActionListener(l: Unit -> Unit \ {ef - Block}, b: Button): Unit \ IO = {
    ...
    let action = _ -> {f() without Block};
    ...
}

def mkGui(): Gui \ IO = {
    let b = mkButton();
    addActionListener(sayHello, b);    // no error
    addActionListener(_ -> sleep(10)); // error: sleep has the Block effect
    ...
}
```

As a consequence of this design, a GUI built through this API will not freeze due to function calls with a `Block` effect running on the UI thread; the UI thread is nevertheless able to execute other effects such as modifying the interface.

The GUI library and its accompanying demonstrative applications consist of 18 functions in approximately 280 lines of Flix code. They are available in the extended version of this paper.

6.2 Event Bus

A second demonstration is an event bus implementation—a simple example of the publish–subscribe model. The type of an event bus `EventBus[ef: Eff, a: Type]` has a parameter `ef` indicating the set of effects it may execute when an event is published. This allows an event bus user to forbid undesirable effects in subscribers. For example, a `Block` effect from one subscriber could delay notification to all subsequent subscribers in the queue, and a `Throw` effect from one subscriber could prevent subsequent subscribers from receiving the event altogether; a programmer may wish to prevent either or both of these from occurring in the event bus.

The API provides two functions for controlling the permitted effects of the event bus: `nonBlocking` and `nonThrowing`, which disallow subscribers from executing the `Block` effect or `Throw` effect, respectively. These act as type-level assertions: they require the effect of the event bus to unify with `{ef - Block}` or `{ef - Throw}`, restricting the set of effects the event bus may execute. Similar functions controlling for other effects can easily be written by an application author.

The `addSubscriber` function's signature ensures that the subscriber's effect is permitted by the event bus. The signature can be understood as requiring that the event bus allow the subscriber's effect `ef1`, *in addition to* any other effects `ef2`.

```
/// Ensures that the given eventbus does not block.
def nonBlocking(_: EventBus[{ef - Block}, a]): Unit

/// Ensures that the given eventbus does not throw.
def nonThrowing(_: EventBus[{ef - Throw}, a]): Unit

/// Adds a subscriber to the eventbus.
def addSubscriber(f: a -> Unit \ ef1, b: EventBus[{ef1 + ef2}, a]): Unit

def run(): Unit \ IO = {
    let b = mkEventBus();
    nonBlocking(b);

    addSubscriber(println, b);        // no error
    addSubscriber(_ -> sleep(10), b); // type error: sleep has the Block effect
    ...
}
```

This event bus interface allows the author to make guarantees about its subscribers and the effect of publishing an event. The API provides functions for preventing blocking and throwing, and a programmer can make similar restrictions for any other effects they would like to control, while maintaining indifference to impertinent effects.

The event bus and its accompanying demonstrative application consist of 10 functions in approximately 90 lines of Flix code. They are available in the extended version of this paper.

7 RELATED WORK

In this section, we review prior work on excluding exceptions, stack inspection, type and effect systems, and effect inference with Boolean unification.

From version 11 on, the C++ language[5] supports a noexcept modifier as an annotation on functions that are not supposed to throw exceptions. The purpose of the keyword is to enable compiler optimizations and to support reflection on the potential for a function to throw. This modifier provides no compile-time guarantees with respect to the function's ability to throw an exception. Rather, it replaces an unhandled exception with a call to std::terminate, crashing the program. In particular, the C++ language specification notes that a compiler "is not permitted to reject an expression merely because, when executed, it throws or might throw an exception from a function with a non-throwing exception specification". In contrast, our system guarantees the impossibility of a Throw effect when so annotated, catching violations at compile time rather than crashing at runtime.

The D language[6] supports a nothrow trait on function definitions which expresses that during the call to the function no exception will be thrown. This property is enforced by the compiler, further supporting the usefulness of effect exclusion. It supports higher-order functions and correctly checks nothrow traits on these. It supports inference of nothrow in some functions. However, it does not support effect polymorphism, nor exclusion of effects other than throwing exceptions, two features which we use in our case studies and which are important for programming with effect exclusion generally.

Stack inspection was a security mechanism that was implemented by the CLR [Fournet and Gordon 2002] and the JVM [Wallach et al. 2000]. Its purpose is to prevent untrusted code from performing dangerous effects. To this end, before an effect would be performed, the runtime stack would be inspected in order to find out whether the effect is allowed to be performed. This is similar in our operational semantics, which dynamically finds all without-frames on the stack. However, stack inspection allows for more complicated rules to be enforced than simply forbidding effects.

Pottier et al. [2005] present a type and effect system for a language with stack inspection that statically guarantees that dynamic checks never fail. Their type system is based on row types [Rémy 1992] and as such can express both the presence and the absence of effects. However, they do not offer a term-level construct like our without that can forbid a certain effect, while allowing all others. Moreover, as usual with stack inspection, even if an effect is forbidden, it can still be enabled locally. Their language features higher-order functions, effect polymorphism, and complete type inference. It is unclear to us if their approach could be extended to allow for programming with effect exclusion.

Type and effect systems [Lucassen and Gifford 1988] have been used successfully to track which operations a computation may perform. Recently, languages with handlers of algebraic effects [Brachthäuser et al. 2020; Hillerström and Lindley 2016; Leijen 2017; Lindley et al. 2017] come with effect systems that guarantee effect safety: whenever an effect is performed there must be a corresponding handler on the stack. This is similar to checked exceptions [Leroy and Pessaux 2000], for example in Java [Zhang et al. 2016]. Our guarantee of effect safety is different: whenever an effect is performed there is no corresponding without-frame on the stack. This enables effect exclusion. While the type and effect system presented by Hillerström and Lindley [2016] can in principle express the absence of effects, they do not state any properties that use this absence, nor do they encourage users to program with effect exclusion. Negative adjustments [Convent et al.

[5] https://en.cppreference.com/w/cpp/11
[6] https://dlang.org/spec/spec.html

2020], which have been invented as a solution to the effect pollution problem, can not be used for effect exclusion as it is easy to circumvent them with adaptors.

Our proof technique of establishing invariants about the abstract machine by encoding them in the typing of stacks is inspired by Brachthäuser et al. [2022]. While they use the same technique to express which effects a stack *handles*, in our paper stack typing expresses which effects it *forbids*.

The Hindley–Milner type system was pioneered by Hindley [1969] and Milner [1978] and later studied by Damas [1984]. Today, the system is used as the theoretical foundation for several real-world programming languages, including Haskell, OCaml, and Standard ML.

Boole undertook early work on Boolean unification and presented a version of the Successive Variable Elimination (SVE) algorithm for the single variable case [Boole 1847]. Later, Löwenheim [1908] presented an alternative algorithm for Boolean unification. Subsequent work includes that of Rudeanu [1974] and Buttner and Simonis [1987]. Martin and Nipkow [1989] provides an accessible introduction to Boolean unification. Boudet et al. [1989] study Boolean unification in combination with other theories, while Baader [1998] study the computational complexity of Boolean unification.

We believe that the first *proposal* to use Boolean unification for type inference dates back to Andrew Kennedy, who proposed it in a technical report [Kennedy 1996]. In his report, Kennedy sketches two applications: one for typing records and one for a region-based effect system. It appears that Boolean unification was first *used* in a type system; to support type inference, by de Vries et al. who used it model uniqueness [de Vries et al. 2008]. In their system, Boolean formulas express uniqueness attributes where true means a variable is unique, false means a variable is non-unique, and a formula describes under what conditions a variable is unique. The authors implement their system in the Morrow compiler.

Madsen and van de Pol [2021] introduce the idea of relational nullable types: where a nullable type system tracks whether an expression may evaluate to null based on its type, a *relational* nullable type system tracks whether an expression may evaluate to null based on its type and the types of other related expressions. For example, if a function expects two arguments: username and password, then a relational nullable type system can capture that either (i) both arguments must be null, or (ii) both arguments must be non-null; i.e., it is not permitted to have username be null while password is non-null. The type system models nullability with two Boolean formulas that capture when an expression may evaluate to null and when it may evaluate to non-null. The system relies on Boolean unification to solve constraints over these formulas.

Madsen and van de Pol [2020] present a polymorphic type and effect system that uses Boolean formulas to describe effects and Boolean unification to support type inference. In the system, the effect formula true represents a pure expression, false represents an expression with possible side-effects, and a formula captures effect polymorphism, i.e., under which conditions a higher-order function is pure. The work of Madsen and van de Pol is close to ours; both present polymorphic type and effect systems based on Hindley–Milner extended with Boolean unification over a Boolean algebra. But there are also significant differences: (i) The focus of their work is to reason about purity, whereas the focus of our work is to support effect exclusion motivated by our case study. (ii) Their effect system has a single effect: impurity, whereas our effect system supports arbitrary user-defined effects. (iii) We introduce a calculus with the without construct and prove that the effect system enforces that a forbidden effect cannot occur at runtime. (iv) A technical detail in the implementation is that we use a symbolic top element to support an open effect universe to enable modularity. (v) Finally, we believe our proposed effect system is well-positioned to support algebraic effects in the future.

8 CONCLUSION

Programmers are often required to reason about the absence of effects. Yet, most programming languages only directly support reasoning about the potential presence of effects. Reasoning about the absence is only achieved indirectly by manually computing the complement. This mode of reasoning, however, breaks down when also considering effect polymorphism.

In this paper, we presented a calculus $\lambda_{\complement}$ that enables programmers to directly program and reason about *effect exclusion*. Programmers can use the without-construct to express that a context forbids a set of effects. Operationally, performing an effect checks whether the effect is allowed by the current call stack. The type system of $\lambda_{\complement}$ moves this check from run time to compile time. It is based on Hindley-Milner and supports complete type and effect inference via an extension of Algorithm W with Boolean unification of set formulas. The soundness proofs of $\lambda_{\complement}$ include our main theorem of effect safety, which ensures that an effect will never be performed if it is forbidden by the current context.

We have implemented the system as an extension of the Flix programming language, which is open-source, ready for use, and freely available. We conducted a case study of open-source code on GitHub, which identified 59 program fragments where effect exclusion is required for correctness, and we have shown that these program fragments can be recast as Flix programs where the effect exclusion is statically enforced by the type and effect system. This demonstrates the applicability and usefulness of effect exclusion.

As proposed in this paper, we treat the semantics of effects as external to the $\lambda_{\complement}$ calculus. This allows us to focus on tracking the absence of effects in the type of functions, providing additional static guarantees to programmers. In the future, we would like to extend $\lambda_{\complement}$ to fully support algebraic effects and handlers, equipping effect operations with user-definable operational semantics.

ACKNOWLEDGMENTS

The work on this project was supported by the Deutsche Forschungsgemeinschaft (DFG – German Research Foundation), project number DFG-448316946.

REFERENCES

Franz Baader. 1998. On the complexity of Boolean unification. *Inform. Process. Lett.* 67, 4 (1998). https://doi.org/10.1016/s0020-0190(98)00106-9

Andrej Bauer and Matija Pretnar. 2015. Programming with algebraic effects and handlers. *Journal of Logical and Algebraic Methods in Programming* 84, 1 (2015). https://doi.org/10.1016/j.jlamp.2014.02.001

George Boole. 1847. *The mathematical analysis of logic.* Macmillan, Barclay and Macmillan.

Alexandre Boudet, Jean-Pierre Jouannaud, and Manfred Schmidt-Schauss. 1989. Unification in Boolean Rings and Abelian Groups. *Journal of Symbolic Computation* 8, 5 (1989). https://doi.org/10.1016/s0747-7171(89)80054-9

Jonathan Immanuel Brachthäuser, Philipp Schuster, Edward Lee, and Aleksander Boruch-Gruszecki. 2022. Effects, capabilities, and boxes: from scope-based reasoning to type-based reasoning and back. *Proceedings of the ACM on Programming Languages* 6, OOPSLA1 (2022). https://doi.org/10.1145/3527320

Jonathan Immanuel Brachthäuser, Philipp Schuster, and Klaus Ostermann. 2020. Effects as capabilities: effect handlers and lightweight effect polymorphism. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020). https://doi.org/10.1145/3428194

Wolfram Buttner and Helmut Simonis. 1987. Embedding boolean expressions into logic programming. *Journal of Symbolic Computation* 4, 2 (1987). https://doi.org/10.1016/s0747-7171(87)80065-2

Lukas Convent, Sam Lindley, Conor McBride, and Craig McLaughlin. 2020. Doo bee doo bee doo. *Journal of Functional Programming* 30 (2020). https://doi.org/10.1017/s0956796820000039

Luis Damas. 1984. *Type assignment in programming languages.* Ph. D. Dissertation. The University of Edinburgh.

Edsko de Vries, Rinus Plasmeijer, and David M. Abrahamson. 2008. Uniqueness Typing Simplified. In *Implementation and Application of Functional Languages.* https://doi.org/10.1007/978-3-540-85373-2_12

Cédric Fournet and Andrew D. Gordon. 2002. Stack inspection. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages.* https://doi.org/10.1145/503272.503301

James Gosling, Bill Joy, and Guy Steele. 1996. *The Java language specification*. Addison-Wesley Professional.
Daniel Hillerström and Sam Lindley. 2016. Liberating effects with rows and handlers. In *Proceedings of the 1st International Workshop on Type-Driven Development*. https://doi.org/10.1145/2976022.2976033
Roger Hindley. 1969. The principal type-scheme of an object in combinatory logic. *Transactions of the American Mathematical Society (AMS)* (1969).
Ohad Kammar and Matija Pretnar. 2017. No value restriction is needed for algebraic effects and handlers. *Journal of Functional Programming* 27 (2017). https://doi.org/10.1017/s0956796816000320
Andrew J. Kennedy. 1996. *Type inference and equational theories*. Technical Report. École polytechnique, Paris, France.
Daan Leijen. 2014. Koka: Programming with Row Polymorphic Effect Types. (2014). https://doi.org/10.48550/ARXIV.1406.2061
Daan Leijen. 2017. Type directed compilation of row-typed algebraic effects. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. https://doi.org/10.1145/3009837.3009872
Xavier Leroy and François Pessaux. 2000. Type-based analysis of uncaught exceptions. *ACM Transactions on Programming Languages and Systems* 22, 2 (2000). https://doi.org/10.1145/349214.349230
Paul Blain Levy, John Power, and Hayo Thielecke. 2003. Modelling environments in call-by-value programming languages. *Information and Computation* 185, 2 (2003). https://doi.org/10.1016/s0890-5401(03)00088-9
Sam Lindley, Conor McBride, and Craig McLaughlin. 2017. Do be do be do. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. https://doi.org/10.1145/3009837.3009897
Leopold Löwenheim. 1908. *Über das Auflösungsproblem im logischen Klassenkalkul*.
John M. Lucassen and David K. Gifford. 1988. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. https://doi.org/10.1145/73560.73564
Magnus Madsen and Jaco van de Pol. 2020. Polymorphic types and effects with Boolean unification. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020). https://doi.org/10.1145/3428222
Magnus Madsen and Jaco van de Pol. 2021. Relational nullable types with Boolean unification. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (2021). https://doi.org/10.1145/3485487
Urusula Martin and Tobias Nipkow. 1989. Boolean unification — The story so far. *Journal of Symbolic Computation* 7, 3-4 (1989). https://doi.org/10.1016/s0747-7171(89)80013-6
Robin Milner. 1978. A theory of type polymorphism in programming. *J. Comput. System Sci.* 17, 3 (1978). https://doi.org/10.1016/0022-0000(78)90014-4
Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. 1999. Type and Effect Systems. In *Principles of Program Analysis*. https://doi.org/10.1007/978-3-662-03811-6_5
François Pottier, Christian Skalka, and Scott Smith. 2005. A systematic approach to static access control. *ACM Transactions on Programming Languages and Systems* 27, 2 (2005). https://doi.org/10.1145/1057387.1057392
Didier Rémy. 1992. Typing record concatenation for free. In *Proceedings of the 19th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. https://doi.org/10.1145/143165.143202
Sergiu Rudeanu. 1974. *Boolean functions and equations*.
Mads Tofte and Jean-Pierre Talpin. 1997. Region-Based Memory Management. *Information and Computation* 132, 2 (1997). https://doi.org/10.1006/inco.1996.2613
Dan S. Wallach, Andrew W. Appel, and Edward W. Felten. 2000. SAFKASI. *ACM Transactions on Software Engineering and Methodology* 9, 4 (2000). https://doi.org/10.1145/363516.363520
Andrew K. Wright. 1995. Simple imperative polymorphism. *LISP and Symbolic Computation* 8, 4 (1995). https://doi.org/10.1007/bf01018828
Andrew K. Wright and Matthias Felleisen. 1994. A Syntactic Approach to Type Soundness. *Information and Computation* 115, 1 (1994). https://doi.org/10.1006/inco.1994.1093
Yizhou Zhang, Guido Salvaneschi, Quinn Beightol, Barbara Liskov, and Andrew C. Myers. 2016. Accepting blame for safe tunneled exceptions. *ACM SIGPLAN Notices* 51, 6 (2016). https://doi.org/10.1145/2980983.2908086

Received 2023-03-01; accepted 2023-06-27